

Formal Semantics for Multi-Agent Contract-Based Coordination

Agent Runtime Team

February 16, 2026

1 Introduction

We present a formal model for agent runtime contracts using the π -calculus, a process algebra for concurrent systems with mobile processes. Our model formalizes the composition of heterogeneous agent plugins and proves key safety properties (deadlock-freedom, progress guarantees).

2 Syntax

2.1 Contract Channels

Each contract type is modeled as a typed channel with input/output capabilities:

Contract Types: $\tau ::=$ `ToolContract(tool_name, args, result)`
`|MessageContract(target, msg, ack)`
`|SharedContext(key, value)`
`|RecorderContract(event)`
`|ControlContract(signal)`

2.2 Process Terms

Agents and contracts are modeled as concurrent processes:

$P, Q ::= \mathbf{0}$	(nil process)
$ c!\langle v \rangle.P$	(output on channel c)
$ c?(x).P$	(input on channel c , bind to x)
$ P \mid Q$	(parallel composition)
$ \nu c.P$	(channel restriction)
$!P$	(replication)
$ \tau.P$	(internal action)

2.3 Agent Model

An agent is a process composed of multiple contract channels:

$$\text{Agent}_i = \nu c_1, \dots, c_n. (\text{ToolContract}(c_1) \\ | \text{MessageContract}(c_2) \\ | \text{SharedContext}(c_3))$$

2.4 Contract Implementations

Each contract is modeled as a replicated server process:

ToolContract

$$\text{ToolContract}(c) = !c?(tool, args, ret). \tau.ret! \langle \text{execute}(tool, args) \rangle$$

MessageContract

$$\text{MessageContract}(c) = !c?(target, msg, ack). \tau.ack! \langle \text{sent} \rangle$$

SharedContext

$$\begin{aligned} \text{SharedContext}(c) &= !\text{Store}(store) \\ \text{Store}(store) &= c.get?(key, ret).ret! \langle store[key] \rangle. \text{Store}(store) \\ &\quad + c.set?(key, val). \text{Store}(store \cup \{key \mapsto val\}) \end{aligned}$$

3 Operational Semantics

3.1 Structural Congruence

$$\begin{aligned} \overline{P \mid \mathbf{0}} &\equiv \overline{P} & \overline{P \mid Q} &\equiv \overline{Q \mid P} & \overline{(P \mid Q) \mid R} &\equiv \overline{P \mid (Q \mid R)} & \overline{\nu c. \mathbf{0}} &\equiv \mathbf{0} \\ & & & & \frac{c \notin fn(P)}{\nu c.(P \mid Q) \equiv P \mid \nu c.Q} \end{aligned}$$

3.2 Reduction Rules

$$\begin{array}{c}
\text{COMM} \\
\hline
c!\langle v \rangle.P \mid c?(x).Q \rightarrow P \mid Q\{v/x\}
\end{array}
\quad
\begin{array}{c}
\text{PAR} \\
\hline
P \rightarrow P' \quad P \mid Q \rightarrow P' \mid Q
\end{array}
\quad
\begin{array}{c}
\text{RES} \\
\hline
P \rightarrow P' \quad \nu c.P \rightarrow \nu c.P'
\end{array}$$

$$\begin{array}{c}
\text{STRUCT} \\
\hline
P \equiv P' \quad P' \rightarrow Q' \quad Q' \equiv Q \\
\hline
P \rightarrow Q
\end{array}$$

4 Contract Composition

4.1 Multi-Agent System

A multi-agent system is the parallel composition of agents with shared contracts:

$$\begin{aligned}
\text{MAS} = & \nu \text{shared}. (\text{Agent}_1(\text{shared}) \\
& \mid \text{Agent}_2(\text{shared}) \\
& \mid \text{SharedContext}(\text{shared}))
\end{aligned}$$

4.2 Communication Pattern

Agent communication via MessageContract:

$$\begin{aligned}
\text{Agent}_A &= \text{msg}_c!\langle B, \text{"task"} \rangle. \text{ack?}(\text{resp}). \text{Continue} \\
\text{Agent}_B &= \text{msg}_c?(sender, task). \text{process}(task). \text{ack!}(\text{"done"})
\end{aligned}$$

5 Type System

5.1 Channel Types

Channels are typed to ensure correct usage:

$$\Gamma ::= \emptyset \mid \Gamma, c : T$$

where T is a channel type:

$$\begin{array}{ll}
T ::= & \uparrow \tau \quad (\text{output capability}) \\
& \mid \downarrow \tau \quad (\text{input capability}) \\
& \mid \uparrow \tau \otimes \downarrow \tau \quad (\text{bidirectional})
\end{array}$$

5.2 Typing Rules

$$\begin{array}{c}
\text{T-OUT} \\
\frac{\Gamma \vdash v : \tau \quad \Gamma \vdash P}{\Gamma, c : \uparrow \tau \vdash c!\langle v \rangle.P}
\end{array}
\quad
\begin{array}{c}
\text{T-IN} \\
\frac{\Gamma, x : \tau \vdash P}{\Gamma, c : \downarrow \tau \vdash c?(x).P}
\end{array}
\quad
\begin{array}{c}
\text{T-PAR} \\
\frac{\Gamma_1 \vdash P \quad \Gamma_2 \vdash Q}{\Gamma_1 \circ \Gamma_2 \vdash P \mid Q}
\end{array}$$

6 Safety Properties

6.1 Theorem 1: Deadlock-Freedom

Theorem 1 (Deadlock-Freedom). *Let $MAS = Agent_1 \mid \dots \mid Agent_n \mid Contracts$ be a well-typed multi-agent system. Then MAS is deadlock-free, i.e., if $MAS \not\rightarrow$ then $MAS \equiv \mathbf{0}$.*

Proof Sketch. We prove by induction on the structure of MAS :

Base Case: If $MAS = \mathbf{0}$, then trivially deadlock-free.

Inductive Case: Assume $MAS = P \mid Q$ where P, Q are deadlock-free.

1. **Contract Channels are Acyclic:** By construction, contract dependencies form a DAG:
 - Agents call contracts (agent $\rightarrow$ contract)
 - Contracts never call other contracts directly
 - No cyclic dependencies: agent $\not\rightarrow$ contract $\not\rightarrow$ agent
2. **All Contract Calls are Replicated Servers:** Each contract is defined as $!c?(x).P$, meaning it can always receive messages. Therefore, no agent can block waiting for a contract.
3. **SharedContext is Lock-Free in Model:** In our π -calculus model, SharedContext operations are atomic messages (no explicit locks). The implementation uses `asyncio.Lock` but the abstract model is lock-free.
4. **MessageContract is Asynchronous:** Message sending does not block (fire-and-forget). Even with acknowledgments, the ack channel is provided by caller, so no circular wait.

Therefore, MAS cannot deadlock unless agents create cyclic dependencies, which our type system prevents via acyclicity constraint. $\square$

6.2 Theorem 2: Progress

Theorem 2 (Progress). *Let MAS be a well-typed multi-agent system. If an agent calls a contract method, the contract responds within a finite number of reduction steps (assuming non-diverging contract implementations).*

Proof Sketch. By induction on the structure of contract calls:

ToolContract:

$$\begin{aligned} \text{Agent} &= \text{tool}_c! \langle \text{"api"}, \text{args}, \text{ret} \rangle . \text{ret?}(\text{result}).P \\ \text{ToolContract} &= !\text{tool}_c?(\text{tool}, \text{args}, \text{ret}'). \tau . \text{ret}'! \langle r \rangle \end{aligned}$$

Reduction sequence:

$$\begin{aligned} &\text{Agent} \mid \text{ToolContract} \\ &\rightarrow \text{ret?}(\text{result}).P \mid \tau . \text{ret}'! \langle r \rangle && \text{(by Comm)} \\ &\rightarrow \text{ret?}(\text{result}).P \mid \text{ret}'! \langle r \rangle && \text{(by internal action)} \\ &\rightarrow P\{r/\text{result}\} && \text{(by Comm)} \end{aligned}$$

Progress guaranteed in 3 steps (assuming τ terminates).

SharedContext: Get operation:

$$\begin{aligned} &c_get! \langle \text{key}, \text{ret} \rangle . \text{ret?}(\text{val}).P \mid \text{Store}(\text{store}) \\ &\rightarrow \text{ret?}(\text{val}).P \mid \text{ret}'! \langle \text{store}[\text{key}] \rangle . \text{Store}(\text{store}) \\ &\rightarrow P\{\text{store}[\text{key}]/\text{val}\} \mid \text{Store}(\text{store}) \end{aligned}$$

Progress in 2 steps.

Therefore, all contract calls make progress within finite steps. $\square$

6.3 Theorem 3: Sequential Consistency for SharedContext

Theorem 3 (Sequential Consistency). *SharedContext operations appear to execute atomically in some sequential order consistent with the program order of each agent.*

Proof Sketch. Our SharedContext model is:

$$\begin{aligned} \text{Store}(\text{store}) &= c_get?(k, \text{ret}). \text{ret}'! \langle \text{store}[k] \rangle . \text{Store}(\text{store}) \\ &\quad + c_set?(k, v). \text{Store}(\text{store} \cup \{k \mapsto v\}) \end{aligned}$$

Key Observation : The store is passed as a functional argument, creating a sequential chain:

$$\text{Store}(s_0) \rightarrow \text{Store}(s_1) \rightarrow \text{Store}(s_2) \rightarrow \dots$$

Each state transition is atomic (single reduction step), and the order is determined by the reduction strategy (e.g., left-to-right).

Implementation Note : The Python implementation uses `asyncio.Lock`, which provides mutual exclusion. The π -calculus model abstracts this as atomic state transitions.

Therefore, SharedContext maintains sequential consistency. $\square$

7 Limitations of Current Model

7.1 What We Model

- Contract interfaces (channels with typed messages)
- Synchronous request-response patterns
- Atomic SharedContext operations
- Deadlock-freedom via acyclic dependencies

7.2 What We Don't Model (Yet)

- **Timeouts:** Need timed π -calculus (π_t -calculus)
- **Failures:** Need failure calculus (crash/Byzantine)
- **Distributed Coordination:** Need distributed π -calculus with locations
- **Priority Queues:** MessageContract has priorities, not modeled here
- **TTL Expiry:** SharedContext TTL requires timed semantics

7.3 Future Extensions

Timed π -Calculus: Add time annotations to model timeouts and TTL:

$$P ::= \dots \mid \text{delay}(t).P \mid \text{timeout}(t, P, Q)$$

Probabilistic π -Calculus: Model probabilistic contract failures:

$$P ::= \dots \mid P \oplus_p Q \quad (\text{choose } P \text{ with prob } p)$$

Spatial π -Calculus: Model distributed agents with locations:

$$P ::= \dots \mid \text{loc}[P] \quad (\text{process at location})$$

8 Conclusion

We have formalized agent runtime contracts using π -calculus and proved:

1. **Deadlock-Freedom:** Acyclic contract dependencies prevent circular wait
2. **Progress:** All contract calls complete in finite steps
3. **Sequential Consistency:** SharedContext operations are atomic

These guarantees ensure that contract-based multi-agent systems are safe and correct by construction.

Next Steps:

- Formalize timeouts and TTL using timed -calculus
- Model distributed coordination with location calculus
- Implement TLA+ specifications for model checking
- Extend to probabilistic contracts (failure handling)